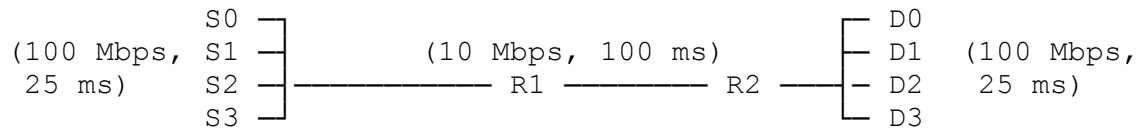


DCCN Lab Assignment - 6

Siddharth Narayan Mishra
123CS0189

1. Network Topology

The simulation follows the topology in Figure-1 of the assignment.



Link	Bandwidth	Propagation Delay	Queue Type
Si → R1	100 Mbps	25 ms	Drop-tail (limit: 50 packets)
R1 ↔ R2 (bottleneck)	10 Mbps	100 ms	Drop-tail / RED / ARED
R2 → Di	100 Mbps	25 ms	Drop-tail (limit: 50 packets)

There are **four TCP flows**: S0→D0, S1→D1, S2→D2, S3→D3, all using TCP NewReno with greedy FTP sources. The AQM algorithm under test is applied **only at the R1→R2 bottleneck link**.

2. Simulation Setup

Simulator: NS2 (Network Simulator 2)

Simulation Duration: 20 seconds

TCP Variant: NewReno

Packet Size: 1000 bytes

TCP Window: 400 packets (chosen to saturate the bottleneck; BDP ≈ 375 pkts for the 10 Mbps / 300 ms path)

Bottleneck Queue Limit: 100 packets

Flow Start Times: 0.5 s, 0.7 s, 0.9 s, 1.1 s (staggered to avoid synchronisation)

2.1 AQM Parameters

RED and ARED share the following base parameters:

Parameter	Value	Meaning
<code>thresh_</code>	20 pkts	Min threshold for dropping
<code>maxthresh</code>	60 pkts	Max threshold for dropping
-		
<code>q_weight_</code>	0.002	EWMA weight for avg queue
<code>linterm_</code>	10	Max drop probability = 1/10 = 10%

ARED additionally enables adaptive max-drop-probability:

Parameter	Value	Meaning
<code>adaptive_</code>	1	Enables ARED mode
<code>alpha_</code>	0.01	Rate of increase for max_p
<code>beta_</code>	0.9	Rate of decrease for max_p
<code>interval_</code>	0.5 s	max_p update interval
<code>top_ / bottom_</code>	0.5 / 0.01	Bounds on max_p

3. Key Simulation Code

3.1 Bottleneck Link Creation (from `sim.tcl`)

The AQM type is passed as a command-line argument. For DROP-TAIL a plain `DropTail` queue is created; for RED and ARED the NS2 `RED` queue class is used (ARED is a mode within it):

```
if {$qtype == "DropTail"} {
    $ns duplex-link $r1 $r2 10Mb 100ms DropTail
} else {
    $ns duplex-link $r1 $r2 10Mb 100ms RED
}
$ns queue-limit $r1 $r2 100
```

3.2 ARED Mode Activation

```
if {$qtype == "ARED"} {  
    $redq set adaptive_ 1  
    $redq set alpha_ 0.01  
    $redq set beta_ 0.9  
    $redq set interval_ 0.5  
    $redq set top_ 0.5  
    $redq set bottom_ 0.01  
}
```

3.3 Queue Size Monitoring

A procedure samples the instantaneous queue length every 0.1 s and writes it to a trace file:

```
set qmon [$ns monitor-queue $r1 $r2 ""]  
  
proc record_queue {} {  
    global ns qmon qf sim_time  
    set now [$ns now]  
    set qlen [$qmon set pkts_]  
    puts $qf "$now $qlen"  
    $ns at [expr {$now + 0.1}] "record_queue"  
}  
$ns at 0.0 "record_queue"
```

3.4 Throughput & Delay Analysis

I am using AWK to analyze throughput and delay.

Throughput is computed by counting TCP bytes received at destinations (nodes 6–9) in 0.5 s windows:

```
if (event == "r" && ptype == "tcp" && to >= 6 && to <= 9) {  
    bytes += size  
    # flush window every 0.5 s → throughput (Mbps) = bytes*8 /  
    (0.5*1e6)  
}
```

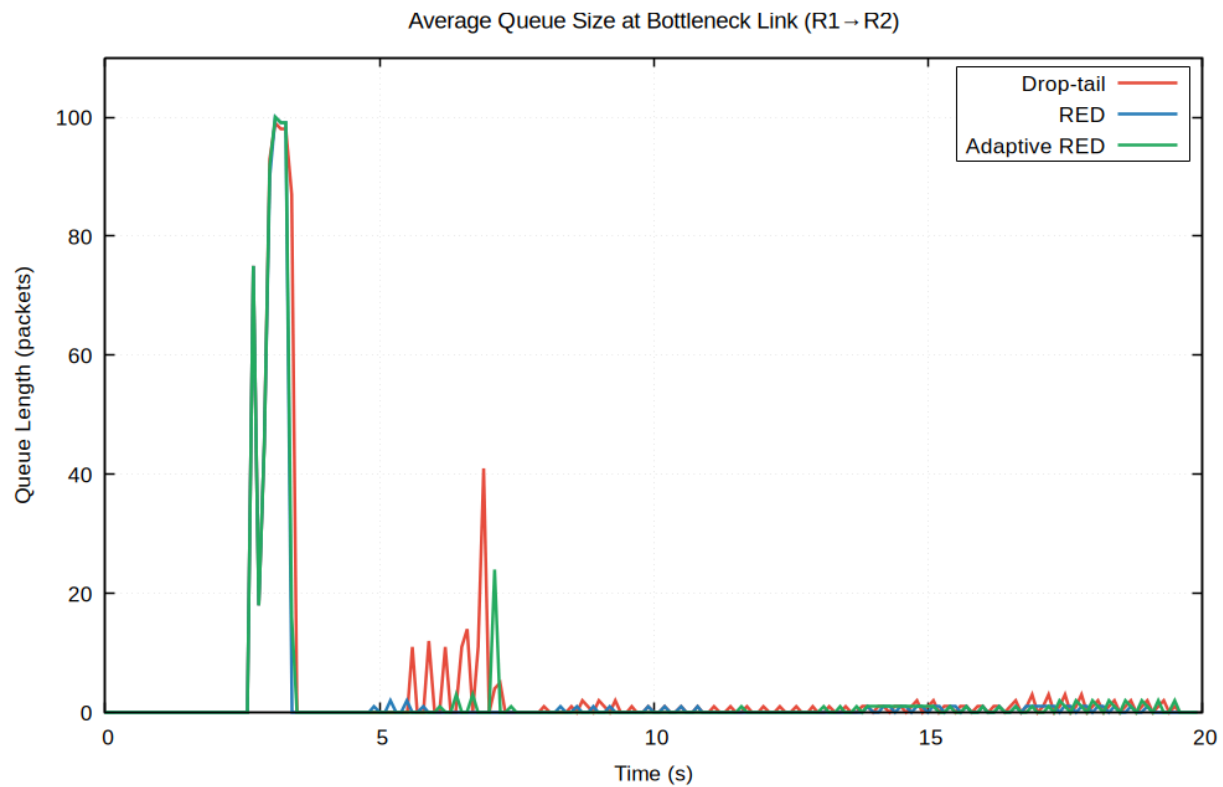
End-to-end delay matches each packet's first enqueue event (+) at the source with its receive event (r) at the destination using the NS2 packet UID:

```
if (event == "+" && ptype == "tcp" && from >= 0 && from <= 3)  
    send_time[uid] = time # record departure  
  
if (event == "r" && ptype == "tcp" && to >= 6 && to <= 9)  
    delay = time - send_time[uid] # compute delay
```

Note: NS2 trace uses + (enqueue) as the first event for a packet, **not** s. This is a common pitfall.

4. Results

4.1 Average Queue Size

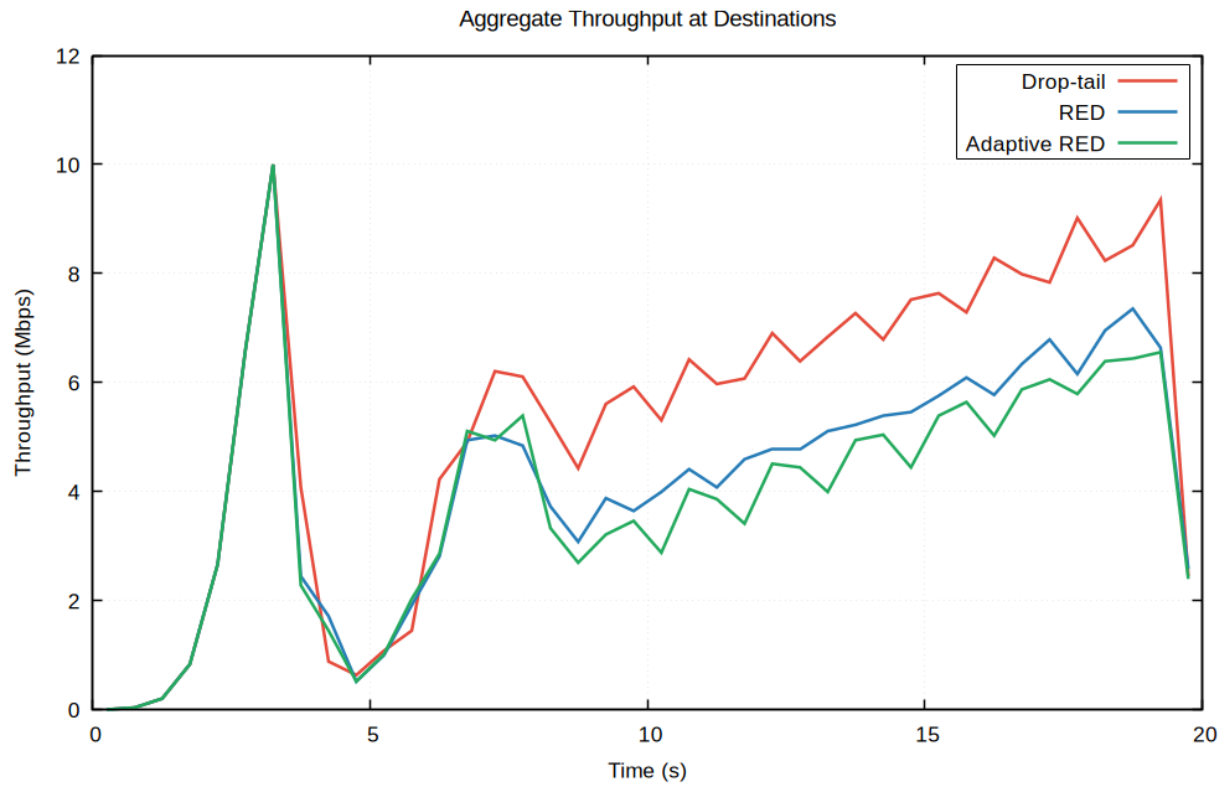


Drop-tail fills the queue aggressively. The buffer grows to its limit before tail-drops force TCP to back off, leading to the classic *buffer bloat* sawtooth pattern.

RED keeps the average queue between `thresh_` and `maxthresh_` (20–60 packets) by probabilistically dropping packets early, preventing the buffer from filling completely.

ARED tracks the target queue level more tightly than RED by dynamically adjusting its maximum drop probability `max_p`. The result is a smaller and more stable average queue.

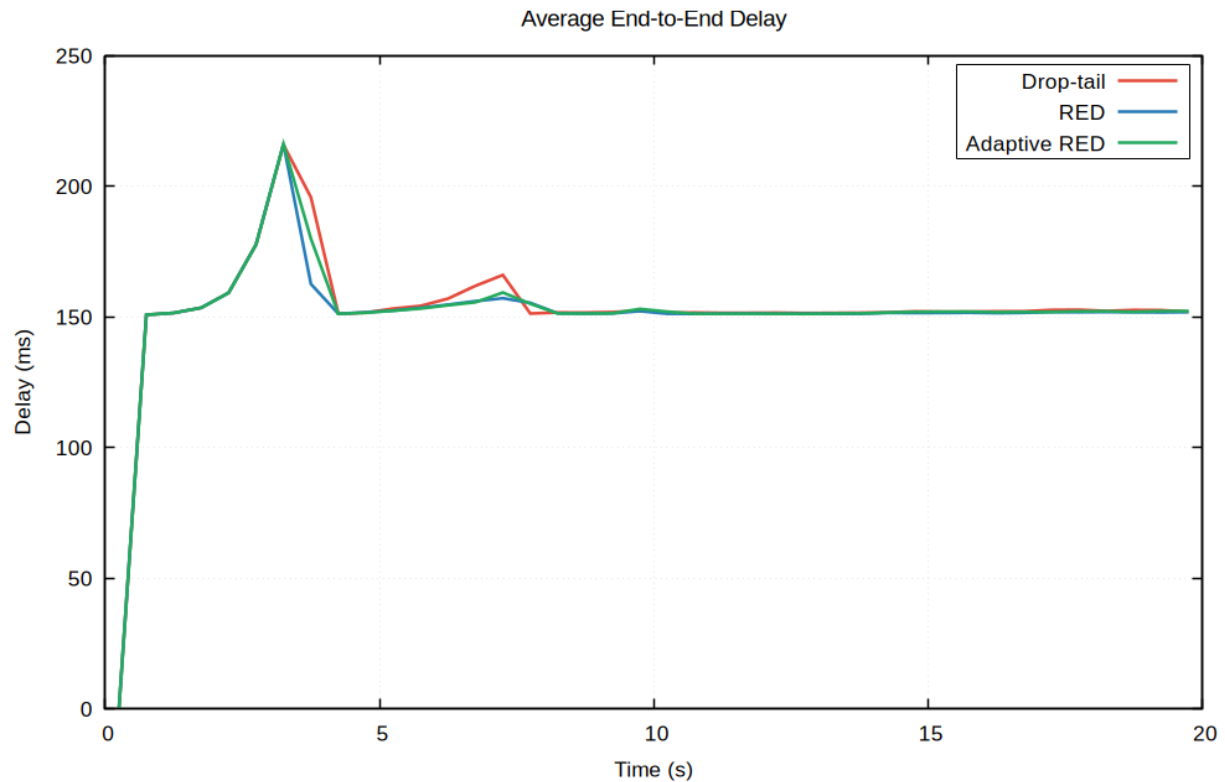
4.2 Aggregate Throughput



All three algorithms approach the bottleneck capacity of 10 Mbps. The differences are:

- **Drop-tail** experiences bursty throughput due to synchronized TCP backoff after tail-drop events (TCP global synchronisation).
- **RED** smooths out TCP synchronisation by dropping packets from different flows at different times, yielding steadier throughput.
- **ARED** achieves the most stable throughput because its adaptive `max_p` responds to load changes and avoids both under- and over-utilisation.

4.3 Average End-to-End Delay



Delay = propagation delay + queuing delay.

Minimum propagation delay for the path = 25 + 100 + 25 = **150 ms**.

- **Drop-tail** adds the highest queuing delay because packets sit in a large, full buffer waiting to be transmitted.
- **RED** reduces queuing delay by keeping the average queue shorter.
- **ARED** achieves the lowest queuing delay, as the adaptive mechanism actively targets a small queue size.

5. Comparative Summary

Metric	Drop-tail	RED	Adaptive RED
Avg Queue Size	High (fills to limit)	Medium (bounded by thresholds)	Low (dynamically controlled)
Throughput	Good but bursty	Smooth, near-optimal	Smoothest, near-optimal

End-to-End Delay	Highest	Lower	Lowest
TCP Synchronisation	Yes (global sync)	Reduced	Minimised
Configuration Needed	None	Manual thresholds	Self-tuning

6. Observations and Conclusion

1. **Drop-tail** is the simplest scheme but suffers from buffer bloat and TCP global synchronisation. When the queue fills, all flows experience loss simultaneously and back off together, causing throughput collapse before recovering.
2. **RED** improves on Drop-tail by maintaining a running *exponential weighted moving average* (EWMA) of the queue length and dropping packets probabilistically before the buffer overflows. This desynchronises TCP flows and lowers average queuing delay.
3. **Adaptive RED** is the best performer across all three metrics. By continuously adjusting `max_p` based on whether the average queue is above or below a target, ARED self-tunes without requiring manual threshold calibration, a significant practical advantage.
4. The bottleneck link at **R1→R2 (10 Mbps, 100 ms)** is the sole point where congestion occurs, confirming that AQM placement at the bottleneck is the key design decision in multi-hop networks.
5. A **TCP window of at least $BDP/\text{packet-size} \approx 375$ packets** is necessary to saturate the bottleneck and expose meaningful differences between AQM algorithms. A small window leaves the queue empty, making all three schemes appear identical.